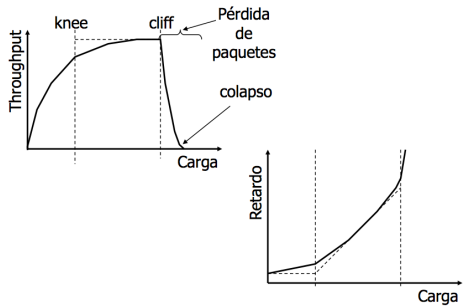
Redes de Datos II
Nivel de Transporte – TCP
Control de Congestión

Luis Marrone

LINTI–UNLP

22 de septiembre de 2026



Facultad de Informática
Facultad de Ingeniería

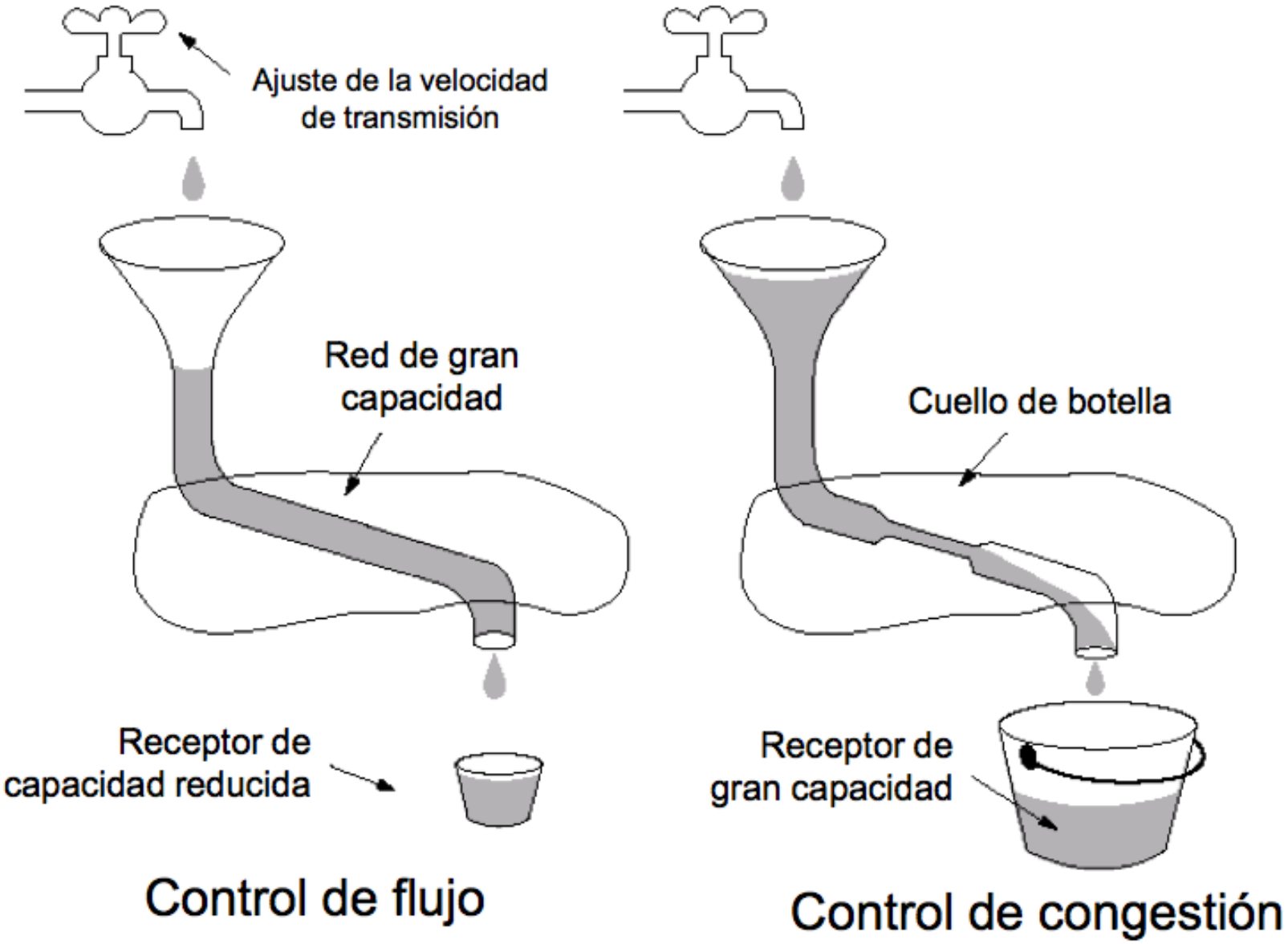
Contenidos

1 Control de Congestión

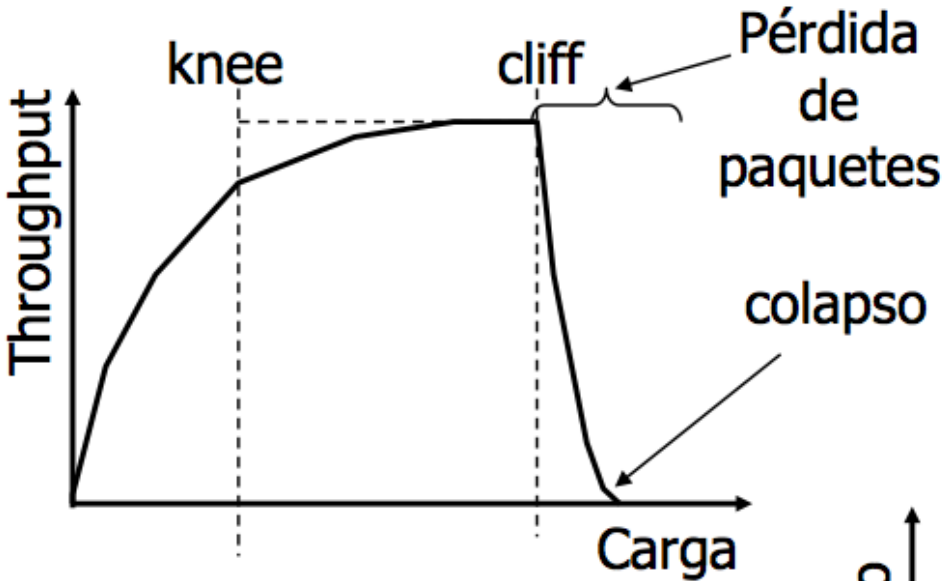
Estamos en:

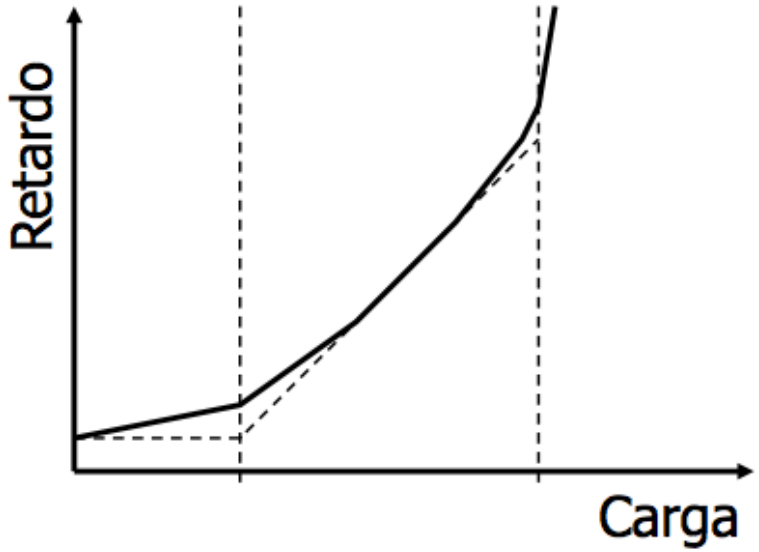
1. Control de Congestión

Control de Flujo vs. Control de Congestión



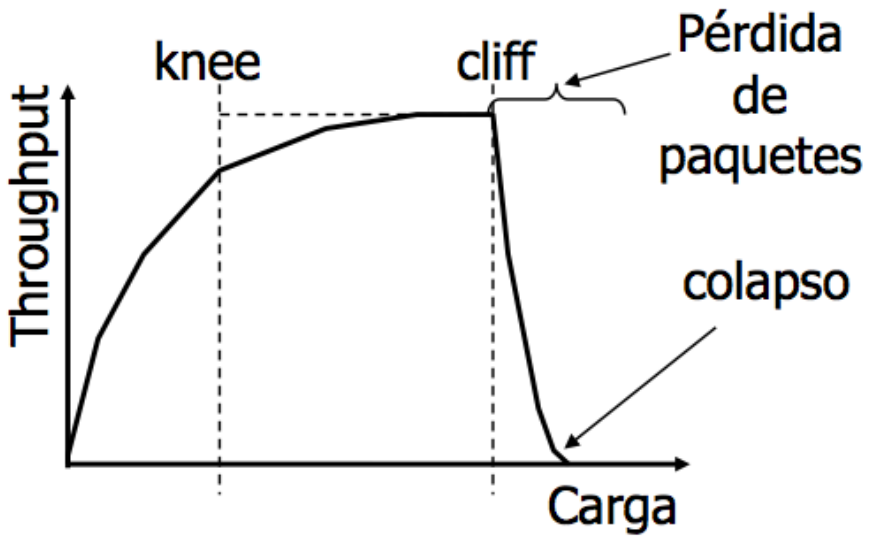
Throughput vs. Congestión





Controlar vs. Evitar

- Controlar
  - Permanecer a la izquierda del *cliff*
- Evitar
  - Permanecer a la izquierda del *knee*



Modelos

- Modelo end-to-end
  - Los extremos son la fuente de la demanda
  - Los extremos deben estimar los tiempos y grado de congestión y reducir la demanda
  - Los nodos intermedios deben monitorear el estado de la red.
- Modelo basado en la red
  - Los extremos no son confiables
  - El nodo de la red tiene control sobre el tráfico
  - Acciones más rápidas

TCP – Modelo end-to-end

- Utiliza tres variables:
  - cwnd: ventana de congestión
  - rcv_win: ventana del receptor. Publicada en el segmento.
  - ssthresh: valor del umbral de *slow start*. Actualiza cwnd.
- Para el envío
  - ventana = min(rcv_win,cwnd)

Slow Start

- ✓ Inicializa el sistema y descubre la congestión rápidamente.
- ✓ Incrementa cwnd hasta la congestión
- ✓ Estima el óptimo cwnd
- ✓ Detecta congestión por pérdida de segmentos
- ✓ Desventajas
  - Detección tardía
  - Enlaces de alta velocidad $--> $ ventanas mayores $--> $ mayor pérdida
  - Interacción con el algoritmo de retransmisión

Slow Start ...

- ✓ En el comienzo o después de congestión

$$cwnd = 1$$

- ✓ Después de cada segmento validado

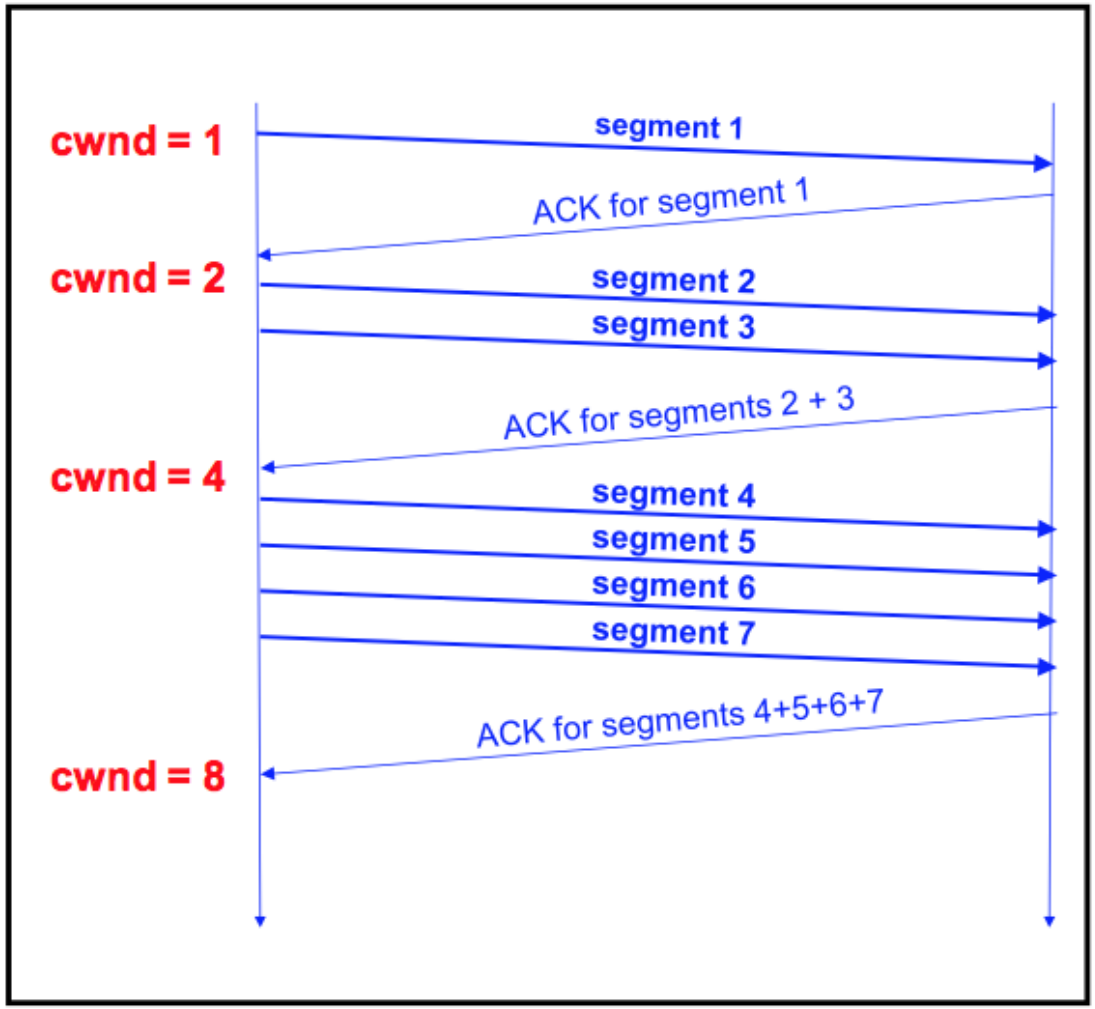
$$cwnd <-- cwnd + 1$$

- ✓ TCP detiene el crecimiento de cwnd cuando

$$cwnd \geq ssthresh$$

- ✓ Pese al incremento unitario resulta exponencial

Slow Start – Ejemplo



Estrategia del control de Congestión

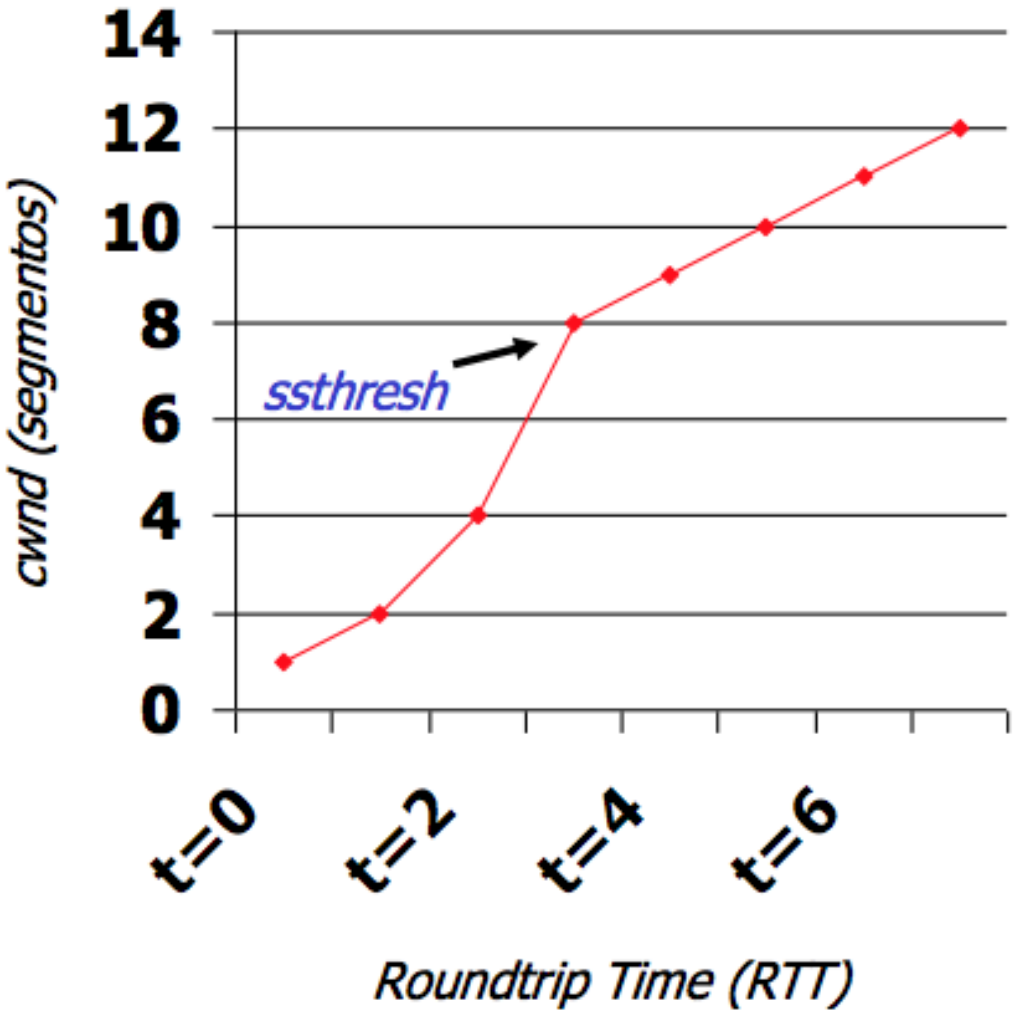
- ✓ Incremento aditivo, comenzar con ssthresh, incrementar cwnd lentamente.
- ✓ Disminución multiplicativa
  Cortar la ventana de congestión drásticamente si se detecta una pérdida.
- ✓ Si *cwnd* $>$ *ssthresh* entonces, por cada ACK

$$cwnd <-- cwnd + \frac{1}{cwnd}$$

- ✓ cwnd se incrementa en 1 si todos los segmentos recibieron su validación.

Combinando...

ssthresh = 8



Armando el Rompecabezas

Inicialmente

$cwnd = 1$

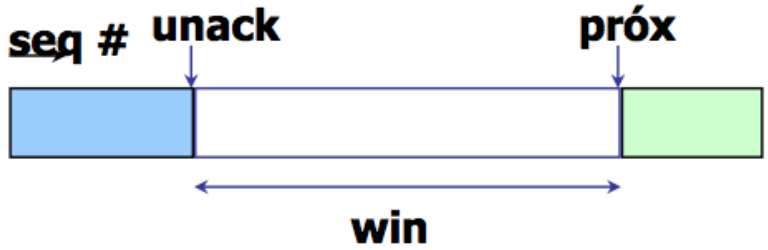
$ssthresh = \infty$

ack recibido
si (cwnd $<$ ssthresh)
/* Slow Start*/
cwnd = cwnd + 1
Tiemout
/* Multiplicative Decrease */

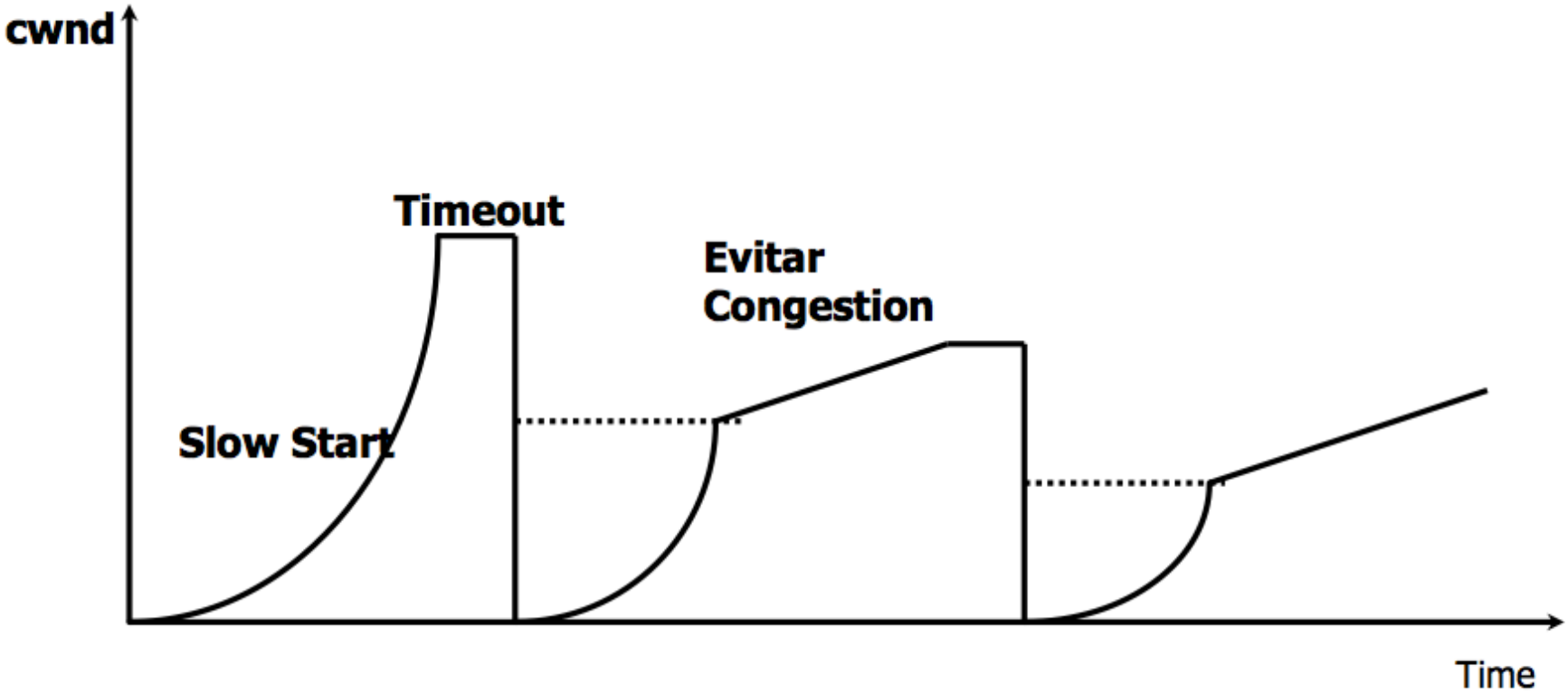
$ssthresh = win/2$

$cwnd = 1$

mientras (próx $<$ unack + win)
transmitir próx segmento
según win = min(cwnd,rcv_win)



Finalmente...



Detección de paquetes perdidos

- ✓ Esperar RTO
- ✓ RTO es usualmente dos veces RTT
- ✓ Degradación de performance
- ✓ No esperar RTO
  - Utilizar mecanismos alternativos
  - Utilizar RTO si no actúan los anteriores

Fast Retransmit y Fast Recovery

- ✓ Frente a un segmento fuera de orden se debe enviar un ACK.
- ✓ Provoca duplicación de ACKs.
- ✓ Esta duplicación es debido a:
  - Paquetes perdidos
  - Reordenamiento de paquetes
- ✓ No se puede discriminar
- ✓ Si se reciben 3 ACKs duplicados se considera que se debe a un paquete perdido
- ✓ Al recibir el tercer ACK repetido se retransmite sin esperar el RTO.
- ✓ Eso es Fast Retransmit
- ✓ Luego se ejecuta *congestion avoidance*?. NO.
  - Slow Start
- ✓ Eso es Fast Recovery.

Integrando...

- ✓ Slow Start.
- ✓ Congestion Avoidance.
- ✓ Si aparecen ACKs duplicados
  Fast Retransmit y Fast Recovery.
  Congestion Avoidance.
- ✓ Si RTO
  Slow Start
- ✓ Resumiendo, TCP Reno.

TCP Vegas

- ✓ 1994
- ✓ Crecimiento más lento que el slow start
- ✓ Nuevo mecanismo de retransmisión
- ✓ Se chequea el TO al recibir el primer ACK duplicado
- ✓ Nuevo algoritmo de congestion avoidance
- ✓ Evita las oscilaciones de Reno
- ✓ Monitorea la diferencia entre el throughput estimado y el real
- ✓ Trata de reducir a cero los paquetes almacenados en los buffers de los routers

Ventana TCP Vegas

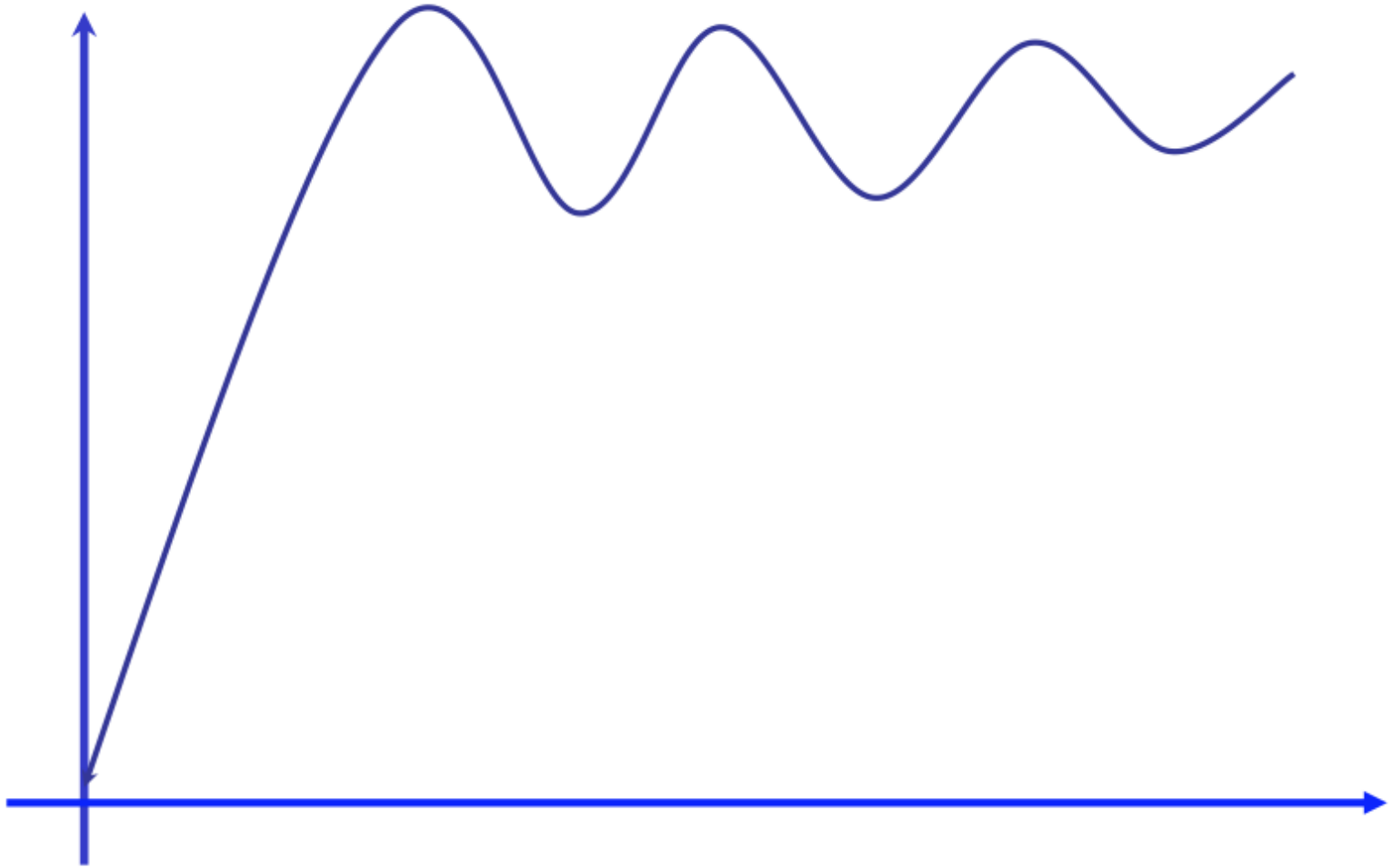
$$w_s(t+1) = \begin{cases} w_s(t) + \frac{1}{D_s(t)}, & si\ \frac{w_s(t)}{d_s} - \frac{w_s(t)}{D_s(t)} < \alpha_s \\ w_s(t) - \frac{1}{D_s(t)}, & si\ \frac{w_s(t)}{d_s} - \frac{w_s(t)}{D_s(t)} > \beta_s \\ w_s(t), & \alpha_s < \frac{w_s(t)}{d_s} - \frac{w_s(t)}{D_s(t)} < \beta \end{cases}$$

$D_s(t) =$ *RTT de la fuente s*

$d_s =$ *mínimo RTT de la fuente s*

α *y* β, *parámetros*

Throughput TCP Vegas





Esta obra está sujeta a la licencia Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0) de Creative Commons.

Para detalle de esta licencia visite https://creativecommons.org/licenses/by-nc-sa/4.0/